University Of Engineering and Technology, Lahore

Department of Cyber Security

Course Name: Programming Fundamentals	**Course Code: CMPE-112L**
Student Name: Hafiz Obaid Ahmed	**Roll No: 2026(s)-CYS-47**
Semester: Spring 2026	**Session: 2026(s)**
Lab/Project #: 15	**CLOs to be covered: CLO3**
Project Title: GUI / Graphics Project — Snake Game	**Teacher Name: Hafiz Muhammad Mubashar**

Lab Evaluation:

CLO3	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no

			understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 15: GUI using Qt Designer — Final Project: 2D Snake Game (Graphics Programming with Pygame)

Lab Goals/Objectives:

- Design and implement a graphical, event-driven 2D application with a real-time game loop.
- Apply grid-based coordinate systems, vector-based movement, and collision detection.
- Practice structuring a GUI/graphics project using state machines (running / game-over / restart).

Equipment Required:

- Personal Computer / Laptop
- Python 3.x
- Pygame library
- Text Editor / IDE (VS Code, PyCharm, or IDLE)

Abstract:

This project presents the end-to-end design, architecture, and implementation of a customized 2D Snake Game built using Python and the Pygame graphics framework. The objective was to build a high-fidelity version of the classic arcade game while adding deterministic difficulty variables — specifically randomized static obstacles (traps) that increase structural complexity. The system manages coordinate spaces, grid alignment, velocity-vector based movement, boundary/collision detection, and a growing list of historical coordinate nodes representing the snake's body. The application demonstrates core computer science and GUI-programming principles including event-driven programming, list/array mutation, real-time collision detection, and state-machine loop design, running at a fixed refresh rate of 15 frames per second.

Introduction & Background:

The Snake Game is a well-known foundational project in software engineering education: an entity (the snake) navigates a bounded 2D space to consume randomly generated food. Eating food increases the snake's length, creating a growing spatial challenge where the player must avoid colliding with the walls, the snake's own tail, or any obstacles placed on the board.

This implementation formalizes a strict coordinate grid (10x10 pixel cells within a 500x500 pixel window) and introduces a procedural obstacle generator that places five static obstacle blocks on the board at the start of each game, without overlapping the snake's starting position.

Technical Stack:

Language: Python 3.x, chosen for its rapid prototyping ability, straightforward procedural style, and dynamic typing.
Graphics/Input Library: Pygame — an open-source multimedia library built on SDL, used here for window/surface management, keyboard event handling, and 2D shape rendering.
Supporting modules: 'sys' (used to cleanly terminate the program window) and 'random' (used to generate pseudo-random grid-aligned coordinates for food and obstacles).

Theory — Mathematical Model:

The game canvas is mapped to a 2D discrete coordinate plane of width W = 500 pixels and height H = 500 pixels. The grid step size (block size) is fixed at 10 pixels, giving (W/10) x (H/10) = 50 x 50 = 2,500 unique grid positions. The coordinate system is zero-indexed, with the top-left corner at (0, 0) and the bottom-right corner at (490, 490).

The snake's body is represented as an ordered list of coordinate tuples, functioning like a queue: Snake List = [(x0, y0), (x1, y1), ..., (x_head, y_head)]. On every game-loop tick, a new head position is calculated from the current velocity vector (vx, vy) and appended to the end of the list; the oldest coordinate (the tail) is then removed from the front — unless food was just eaten, in which case the removal step is skipped for that tick so that the snake's length naturally increases.

Algorithm:

1. Initialize Pygame, set the window to 500x500 pixels, and start the system clock.
2. Define constants: block size = 10 pixels, frame rate = 15 FPS.
3. Place the snake's starting head position at the center of the grid.
4. Generate five static obstacle blocks at random grid-aligned positions, ensuring no overlap with the snake's start position.
5. Generate the first food block at a random grid-aligned position.
6. Enter the main game loop and poll keyboard events each frame.
7. Update the velocity vector (vx, vy) based on arrow-key input, while preventing an immediate 180-degree reversal.
8. Compute the new head position: new_x = x + vx, new_y = y + vy.
9. Check the new position against the window boundaries; if it is out of bounds, trigger Game Over.
10. Check the new position against the obstacle list; if it matches an obstacle, trigger Game Over.
11. Check the distance to the food block; if within one block size, increase the score and snake length, and reposition the food.
12. Append the new head to the snake list, and remove the tail element if the list has grown beyond the current snake length.
13. Clear the screen, draw the obstacles, food, and snake segments, then refresh the display.
14. Limit the loop to 15 frames per second and repeat from step 6 until the game ends.

Lab Work:

Task 1: Complete Snake Game Implementation (Pygame)

```
import pygame
import sys
import random

pygame.init()

# ---------------- Configuration Constants ----------------
WIDTH, HEIGHT = 500, 500
BLOCK = 10
FPS = 15

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (213, 50, 80)
DARK_RED = (139, 0, 0)
GREEN = (0, 200, 0)
BLUE = (50, 153, 213)

win = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Snake Game with Obstacles")

clock = pygame.time.Clock()
font_style = pygame.font.SysFont("bahnschrift", 25)
score_font = pygame.font.SysFont("comicsansms", 25)


def generate_obstacles(count, snake_x, snake_y):
    obstacles = []
    while len(obstacles) < count:
        ox = round(random.randrange(20, WIDTH - 20) / 10.0) * 10
        oy = round(random.randrange(20, HEIGHT - 20) / 10.0) * 10
        if abs(ox - snake_x) > 30 or abs(oy - snake_y) > 30:
            obstacles.append((ox, oy))
    return obstacles


def draw_score(score):
    value = score_font.render("Score: " + str(score), True, BLACK)
    win.blit(value, [0, 0])


def draw_snake(block, snake_list):
    for x, y in snake_list:
        pygame.draw.rect(win, GREEN, [x, y, block, block])


def message(msg, color, y_offset=0):
    mesg = font_style.render(msg, True, color)
    win.blit(mesg, [WIDTH / 6, HEIGHT / 3 + y_offset])
```

```
def main_game_loop():
    game_over = False
    game_close = False

    snake_x = WIDTH / 2
    snake_y = HEIGHT / 2

    velocity_x = 0
    velocity_y = 0

    snake_list = []
    snake_length = 1
    score = 0

    obstacles = generate_obstacles(5, snake_x, snake_y)

    food_x = round(random.randrange(10, WIDTH - 10) / 10.0) * 10
    food_y = round(random.randrange(10, HEIGHT - 10) / 10.0) * 10

    while not game_over:

        while game_close:
            win.fill(WHITE)
            message("You Lost! Score: " + str(score), RED)
            message("Press C-Play Again or Q-Quit", BLACK, 40)
            draw_score(score)
            pygame.display.update()

            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    game_over = True
                    game_close = False
                if event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_q:
                        game_over = True
                        game_close = False
                    if event.key == pygame.K_c:
                        main_game_loop()

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                game_over = True
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_LEFT and velocity_x == 0:
                    velocity_x = -BLOCK
                    velocity_y = 0
                elif event.key == pygame.K_RIGHT and velocity_x == 0:
                    velocity_x = BLOCK
                    velocity_y = 0
                elif event.key == pygame.K_UP and velocity_y == 0:
                    velocity_y = -BLOCK
```

```
                    velocity_x = 0
                elif event.key == pygame.K_DOWN and velocity_y == 0:
                    velocity_y = BLOCK
                    velocity_x = 0

        if snake_x >= WIDTH or snake_x < 0 or snake_y >= HEIGHT or snake_y < 0:
            game_close = True

        snake_x += velocity_x
        snake_y += velocity_y

        win.fill(WHITE)
        pygame.draw.rect(win, RED, [food_x, food_y, BLOCK, BLOCK])

        for ox, oy in obstacles:
            pygame.draw.rect(win, DARK_RED, [ox, oy, BLOCK, BLOCK])
            if snake_x == ox and snake_y == oy:
                game_close = True

        snake_head = (snake_x, snake_y)
        snake_list.append(snake_head)
        if len(snake_list) > snake_length:
            del snake_list[0]

        for segment in snake_list[:-1]:
            if segment == snake_head:
                game_close = True

        draw_snake(BLOCK, snake_list)
        draw_score(score)
        pygame.display.update()

        if abs(snake_x - food_x) < BLOCK and abs(snake_y - food_y) < BLOCK:
            food_x = round(random.randrange(10, WIDTH - 10) / 10.0) * 10
            food_y = round(random.randrange(10, HEIGHT - 10) / 10.0) * 10
            snake_length += 2
            score += 10

        clock.tick(FPS)

    pygame.quit()
    sys.exit()


if __name__ == "__main__":
    main_game_loop()
```

Result:

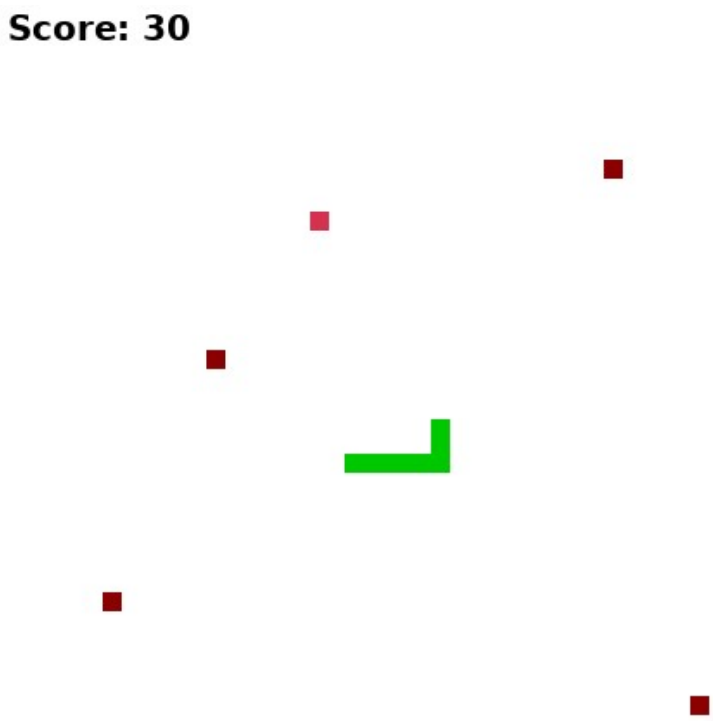

Figure: In-game screenshot showing the snake (green), food (red), static obstacles (dark red), and live score display.

Subsystem Implementation Details:

Tolerance-based food matching: rather than requiring an exact pixel match, food collision is detected using abs(snake_x - food_x) < 10 and abs(snake_y - food_y) < 10, which keeps the game responsive even when the snake changes direction near the food block.

Obstacle isolation: obstacles are generated once at the start of the game and stored in a simple list, so each frame's collision check runs in O(N) time (N = 5), keeping the game loop fast and free of stutter.

Grid alignment: all random coordinates (for food and obstacles) are rounded to multiples of 10 so that every element on screen stays aligned to the same 10-pixel grid as the snake.

Conclusions:

This project demonstrates a clean, event-driven architecture for a 2D arcade game built with Pygame. It applies grid-based coordinate mathematics, real-time keyboard event handling, collision detection against walls/obstacles/self, and a simple state machine to manage the running and game-over states. Future improvements could include persistent high-score storage, a difficulty mode that gradually increases game speed, and procedurally generated obstacle layouts for greater replayability.